Session 2 lab: training an MLP on CIFAR-10

AI Methods for Science (77222) · Thursday 15 October 2026

The setup

You get a working network and a working training procedure: `mlp_lab.py`. One function,

```
result = train(config)
```

takes a dictionary of hyperparameters and returns a dictionary of measurements:

- `result["history"]`: per epoch, `train_loss`, `train_acc`, `val_loss`, `val_acc`, `seconds`;
- `result["summary"]`: `n_params`, `best_epoch` (lowest validation loss), `val_acc_at_best`, `max_val_acc`, final train/validation loss and accuracy, `sec_per_epoch`, `diverged`, and `test_acc` if requested.

The hyperparameters, with defaults (`python mlp_lab.py -show-defaults` prints them):

key	default	meaning
`hidden`	`[512, 256]`	widths of the hidden layers; `[]` is a linear classifier
`activation`	`"relu"`	`"relu"`, `"sigmoid"` or `"tanh"`
`lr`	`0.05`	learning rate of plain SGD
`batch_size`	`128`	minibatch size
`epochs`	`20`	passes over the training set
`train_fraction`	`1.0`	use only this fraction of the 45,000 training images
`normalize`	`true`	standardize the pixels, or feed raw values 0..255
`seed`	`0`	initialization and shuffling (the train/validation split never changes)
`test`	`false`	also report test accuracy. **Once, at the end (E9)**
`device`	`"cpu"`	`"cpu"`, `"cuda"`, `"mps"` or `"auto"`

Your job is the *experiments*: decide what to run, run it, collect the numbers, plot them, and explain what you see.

Rules

- **Do not edit `mlp_lab.py`.** Everyone runs the same network, so tables are comparable across the room.
- **Use an AI assistant for the pipeline** (E0). Writing loops, file handling and plotting code is not the point of the lab.
- **The numbers have to be yours**, measured on your machine. **The explanations have to be yours**: before each experiment, write one line saying what you expect; afterwards, say whether you were right and why.

Before class (at home, not in the classroom)

1. Get the lab folder from the course repository (link on Moodle).
2. You need Python $\geq$ 3.11 with the packages in `requirements.txt` (PyTorch, torchvision, numpy, pandas, matplotlib). Install them with whatever you use: `pip install -r requirements.txt` in a venv or conda environment, or `uv run mlp_lab.py ...`, which installs them by itself. Ask your assistant if unsure.

3. Inside the folder, run
   `python mlp_lab.py -download` (downloads CIFAR-10, 170 MB; can take half an hour on a slow connection)
   `python -c "import mlp_lab; mlp_lab.train({'epochs': 1})"` (one epoch, to check)

Bring a laptop with it working. Times below are for a laptop CPU at ≈ 1.5 s per epoch of the default network. Yours may be 2–3× slower. If so, cut the epochs and say so in your table.

The experiments

Do E0–E4 in class. E5–E9 are for home. Bring your tables and figures to session 3.

E0. Build the pipeline

with your AI assistant, ∼15 min

Ask your assistant for a small tool that

1. takes a named experiment, i.e. a list of configs (e.g. a base config plus one hyperparameter swept over values);
2. runs `mlp_lab.train` on each config and saves every result to disk (one JSON per run);
3. skips runs that are already saved, so an interrupted sweep can be resumed (key the saved file on the whole config, not on a name you type: a key that omits a hyperparameter returns an old run for a new config);
4. keeps a run whose loss became `nan` (its history then holds `None`): record it and show it, do not crash or drop it;
5. collects all runs into one table: config columns + summary columns (a CSV or a pandas DataFrame);
6. plots (a) any `history` curve for several runs on one axis, and (b) any summary quantity against any hyperparameter.

Answer. Check the pipeline against one run you did by hand: same seed, same numbers?

E1. How big is the noise?

∼1.5 min

Run. the default config, `seed` $\in \{0, 1, 2\}$.

Collect. `val_acc_at_best` and `max_val_acc` for each seed.

Answer. How much do they spread? From now on, which difference between two configs would you believe?

E2. The learning rate

∼2 min

Run. `lr` $\in \{0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0\}$, 10 epochs.

Collect. training-loss curves of all runs on one axis (log y); validation accuracy vs. epoch, same runs; `max_val_acc` vs. `lr` (log x).

Answer. Label each run: crawls, works, noisy, diverges. The training loss is an average over the epoch and looks smooth; the noise shows in the validation curve. What is the largest learning rate that does not blow up? Where is the best one relative to it?

E3. Width and depth

∼3 min

Run. `hidden` $\in \{$`[]`, `[64]`, `[512]`, `[2048]`, `[512,256]`, `[1024,1024,1024]`, `[256]*6`$\}$, 15 epochs.

Collect. `n_params`, `sec_per_epoch`, `val_acc_at_best`, `final_train_acc`; plot the last two against `n_params` (log x).

Answer. What does a linear classifier reach? Per parameter, does width or depth buy more? Is any difference you see larger than the seed spread of E1? If not, say so (or run three seeds for the two

networks you want to compare). Does training accuracy predict validation accuracy?

E4. Train for too long ~1.5 min

Run. the default config, 60 epochs.

Collect. train and validation loss vs. epoch; train and validation accuracy vs. epoch.

Answer. At which epoch is the validation loss lowest? What happens to it afterwards, and to the validation *accuracy*? Why can loss and accuracy disagree? Which of the two would you stop on?

E5. Batch size ~2 min

Run. `batch_size` $\in \{16, 128, 1024\}$ at `lr` $= 0.05$; then `batch_size` $= 1024$ at `lr` $= 0.4$. 10 epochs.

Collect. validation accuracy against three different x-axes: epochs, gradient steps (`epoch` $\times$ `steps_per_epoch`), and wall-clock seconds.

Answer. Which batch size wins? The answer depends on the x-axis: explain why. What did raising the learning rate for the large batch do?

E6. How much data ~5 min

Run. `train_fraction` $\in \{0.01, 0.03, 0.1, 0.3, 1.0\}$ with `epochs` $= \min(400, \text{round}(20/\text{fraction}))$, so that the runs take about the same number of gradient steps (the two smallest fractions hit the 400-epoch cap and get fewer steps; note it when you compare).

Collect. `max_val_acc` and `final_train_acc` vs. the number of training images ($\log x$).

Answer. How does the gap between training and validation accuracy depend on the amount of data? Extrapolate: what would $10\times$ more data give?

E7. Activation and depth ~2 min

Run. `activation` $\in$ {relu, sigmoid, tanh} $\times$ `hidden` $\in$ {`[256,256]`, `[256]*8`}, 15 epochs.

Collect. training loss vs. epoch, all six runs on one axis.

Answer. Which combinations train slowly, or not at all? Explain it with Monday's slide on gradients through depth.

E8. No normalization ~1 min

Run. `normalize` $=$ `false`, `lr` $\in \{0.05, 0.001, 0.00001\}$, 10 epochs; compare with the default.

Collect. training loss vs. epoch.

Answer. Why does raw-pixel input fail at the default learning rate? (The first layer's gradient is $\boldsymbol{\delta}^{(1)}\mathbf{x}^\top$.) The loss need not become `nan`: an explosion in epoch 1 followed by a flat $\ln 10 \approx 2.30$, chance level, is the same failure. Can a smaller learning rate alone fix it?

E9. Your best MLP open

Using only the hyperparameters above, find the config with the highest `val_acc_at_best`. Choose it by validation accuracy only. Then run that one config **once** with `test = true` and report the test accuracy. The workshop target (sessions 5–6) is 54% on the test set; with loss-based early stopping the validation number may stay just below it while `max_val_acc` is above. Report both, and say which epoch the test accuracy belongs to.

Answer. Why is the test accuracy run once and not used to choose? Is your test accuracy above or below your validation accuracy, and is the difference within the noise of E1?

What to bring

For each experiment: your one-line prediction, the table, the figure(s), and two or three sentences of explanation. In the last ten minutes of the session we compare tables across the room.